DATA SCIENCE AND AI LAB · MILESTONE 1

Talk to Your Database

A Natural-Language Analytics Copilot

Problem Definition & Literature Review

Team Members:
Siddhant Hitesh Mantri (21f3002218)
Anirudh Komanduri (22f1000522)
Vishal S (23f2003089)
Smrutishikta Das (21f1006009)
Walunila Aier (21f3002564)
Sambhav Jha (22f3003227)

The Problem

- Relational databases hold most of an organization's useful data, but querying them requires SQL knowledge.
- This blocks managers, operations, product teams, and domain experts from directly answering their own data questions.
- **Our solution:** A copilot that turns a plain-English question into a safe, executed SQL query with results shown as a table along with a chart, when appropriate.

CORE WORKFLOW


Understand the question


Retrieve relevant schema


Generate valid SQL


Execute safely (read-only)


Present table + visualization

Scope narrowed per TA feedback: NL→SQL generation, schema-aware retrieval, execution, and visualization are the core deliverables. Self-correction, multi-turn conversations, and multi-model comparison are stretch goals.

Stakeholders & Scope

Primary: non-technical business users & domain experts **Secondary:** analysts, data engineers, DBAs

IN SCOPE

- Single-database querying (SQLite / PostgreSQL)
- Single-turn NL analytical questions
- Read-only SELECT queries
- Schema-aware retrieval (tables, columns, keys)
- SQL retrieval using open-source models (e.g. Phi-4-mini, Qwen-family, etc.)
- SQL validation, row limits, timeouts, read-only execution
- Result display: (SQL, table, auto chart)
- Evaluation using execution accuracy, SQL validity, retrieval quality, latency, and visualization validity

OUT OF SCOPE

- Write ops: INSERT / UPDATE / DELETE / DROP / ALTER
- Multi-database federation
- Production-grade access control
- Training a model from scratch
- Full multi-turn conversation memory
- Large-scale model comparison / benchmarking

Why This Problem Matters

Dashboards fall short

Great for repeated metrics, but can't cover every new question users ask.

Analysts create delay

Custom SQL requests mean waiting - and repeated manual work.

Text-to-SQL is unsolved

Messy schemas, dirty values, domain jargon, ambiguous wording make real DBs hard for text-to-SQL systems.

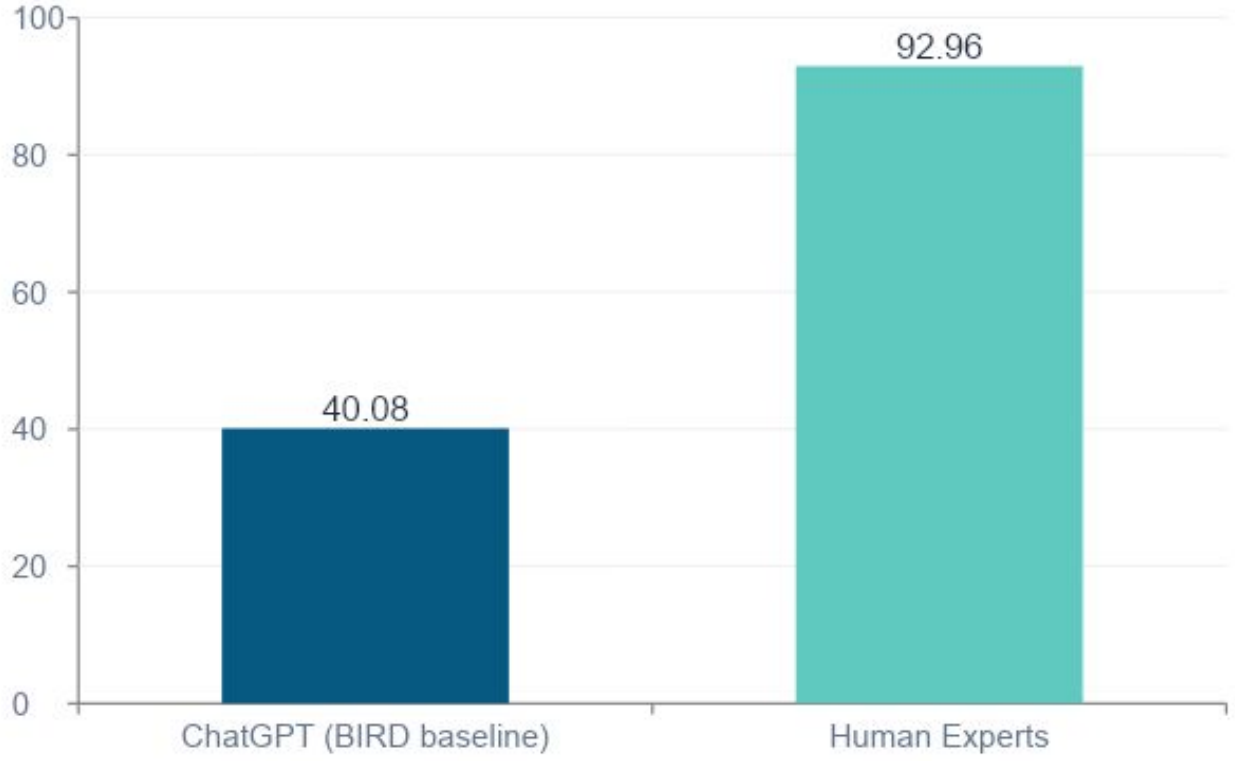
Benchmarks like BIRD and Spider 2.0 exist precisely because older datasets don't understated these real-world difficulties.

Current Approaches and Literature Review

Baseline tools & LLMs	BI tools (Tableau, Power BI, Looker, etc.) and notebooks need SQL/dashboards predefined; general LLMs draft SQL but break without proper schema context.
Early Neural SQL	Seq2SQL/SQLNet (WikiSQL) proved feasibility, but on single-table data - doesn't reflect multi-table org DBs.
Schema Linking	Spider's cross-domain focus highlighted schema linking as a core challenge. RAT-SQL and RESDSQL address it directly, motivating our explicit retrieval module.
Validity & Execution	Execution-guided decoding and PICARD constrain/validate SQL during generation, motivating our SQL validation + safe execution layer.
LLM-era & Self-Correction	Strong LLM baselines; DIN-SQL adds decomposition and self-correction, kept as a stretch goal given added complexity.
Output Usability	nvBench motivates going beyond raw SQL output, supporting our simple rule-based chart selection (metric / bar / line / table).

The Realistic Benchmark Gap

On BIRD (12,751 Q/SQL pairs, 95 databases, 37 domains), execution accuracy shows how far LLM-generated SQL still lags experts.



WHAT THIS MEANS FOR US

A ~53 point gap remains between strong LLM baselines and human experts on realistic, database-grounded questions.

Our system uses smaller open-source models on a simpler demo database - so we set a modest, honest execution-accuracy target rather than promising state-of-the-art results.

KaggleDBQA and Spider 2.0 similarly emphasize messy real data, documentation, enterprise schemas, and workflow complexity

Metrics and Evidence of Success

How we'll measure whether the system works

Our project's success metrics

- **SQL Parse Validity**
- **Read-Only Safety Pass**
- **Execution Success Rate**
- **Execution Accuracy**
- **Schema Retrieval Recall**
- **Median / p95 Latency**
- **Visualization Validity**

Industry-standard metrics referenced

- Exact Match
- Component Matching
- Execution Accuracy
- Test-Suite Accuracy
- Valid Efficiency Score

EVIDENCE

Working FastAPI demo: frontend with user question as input → schema-grounded SQL → safe execution → table + chart.
Baseline prompt-to-SQL vs. schema-aware pipeline evaluated automatically; ~100 examples reviewed manually if needed.

Gaps & Our Contribution

REMAINING GAPS

- Schema linking, value grounding, and ambiguous wording remain unsolved
- Invalid SQL and dialect issues persist across systems
- Benchmarks evaluate SQL alone - real users need the full workflow to a trusted answer
- Agentic systems add accuracy but also complexity, latency, and risk

OUR CONTRIBUTION

Not a new model architecture or state-of-the-art benchmark result

A feasible, transparent analytics interface integrating:

- Schema-aware retrieval
- Read-only SQL generation
- Safe execution
- Automatic visualization

Milestone-Aligned Plan

1. Baseline prompt-to-SQL-to-execution pipeline
2. Test small open-source model baselines
3. Add SQL validation & read-only constraints
4. Add schema-aware retrieval
5. Build FastAPI backend + frontend
6. Add rule-based visualization
7. Evaluate with automated + human-reviewed metrics
8. Stretch: self-correction & multi-turn follow-ups

KEY IMPLEMENTATION DECISIONS

Demo DB: e-commerce SQLite/PostgreSQL (Chinook as fallback)

Baseline Models: Phi-4-mini, Qwen-family small models

Stack: FastAPI backend + frontend

Main risk: LLM inference speed & deployment feasibility